



Amrita School of AI
COIMBATORE CAMPUS

CASE STUDY 1 · BEYOND THE SYLLABUS

Echno

A construction-intelligence platform, and how it is actually deployed.

Dr. Abhijith Anandakrishnan · Assistant Professor
24AIM332 · Odd Semester 2026-27

DISCLOSURE

What this case study is, and is not

This is a system I build and run.

You are getting the **architecture and the engineering decisions**, including the ones that went wrong.

No credentials, no customer data, no commercial terms. Patterns and lessons only. That boundary is itself a professional habit worth copying.

One product, several clients, several environments

A backend, a web application, and mobile clients on iOS and Android.

Plus staging, a marketing site, and per-client deployments.

Two ways to handle that:

copy-paste each environment and keep them in sync forever, or

one definition, many variable sets.

An environment is a set of variables, not a set of scripts.

Everything else in this case study follows from that decision.

Provision, then configure

OpenTofu brings resources into existence:
droplets, firewalls, object storage, DNS records.
Modules, plus one directory per environment.

Ansible makes each host correct:
packages, users, services, TLS, application containers.
One role per stage of the build, each runnable on its
own by tag.

Four commands

```
make plan      ENV=staging-do      # what would change
make apply     ENV=staging-do      # create the infrastructure
make site      ENV=staging-do      # configure the host end to end
make deploy    ENV=staging-do TAGS=apps # just redeploy the applications
make destroy   ENV=staging-do      # remove it, stop billing
```

Everything a person needs to do is one of these. Nothing requires knowing which tool is underneath.

Same roles, three very different targets

ENVIRONMENT	TARGET
staging-do	DigitalOcean droplet, Bangalore
marketing-do	small droplet, static site
staging-iitm	a VPS at IIT Madras

The IIT Madras box has no public IP and is reached through a **Cloudflare Tunnel**.

Same Ansible roles. Different inventory and group variables.

IF THE ROLES HAD HARD-CODED THE CLOUD, THIS WOULD HAVE BEEN A REWRITE

The rule, and how it is enforced

Secrets live in an **Ansible Vault** encrypted file, and are rendered into `.env` files **on the target** by templates.

No application repository holds a real credential.
Nothing unencrypted is ever committed. Rotation is a documented runbook, not tribal knowledge.

Not everywhere, and not first

Single-host environments run **Docker Compose**, generated from templates owned by the role that needs them.

That is enough for staging and for a marketing site.

The Kubernetes track is separate, with **ArgoCD** for GitOps: the cluster pulls its desired state from git rather than a pipeline pushing to it.

RECONCILIATION AGAIN

Why GitOps is different

- **Pipeline pushes**

CI holds cluster credentials and runs `kubectl apply`.

Simple. But CI now has production access, and drift is invisible.

- **Cluster pulls**

The cluster watches a repo and reconciles itself.

CI only writes to git. Drift is corrected automatically, and the repo is always the truth.

The same lesson, one layer up

Domain types, API calls and data hooks used to live inside the web app.

Fine for one app. With iOS and Android arriving, the options were copy-paste or extract.

They were extracted into a shared package consumed by all three clients.

A bug fixed once propagates everywhere on the next version bump.

Same reasoning as one Ansible role for three environments.

The recurring decision: duplicate, or parameterise.

Duplication is faster today and compounds against you forever.

Four transferable points

1. An environment is variables, not scripts.
2. Split provisioning from configuration; they change at different rates.
3. Give operators a tiny command surface. Tools are an implementation detail.
4. Secrets are rendered at deploy time, never committed, rotation documented.

Next: the same problems, on Azure, with a stricter pipeline.